Assignment #5: Strategy Design Pattern Report

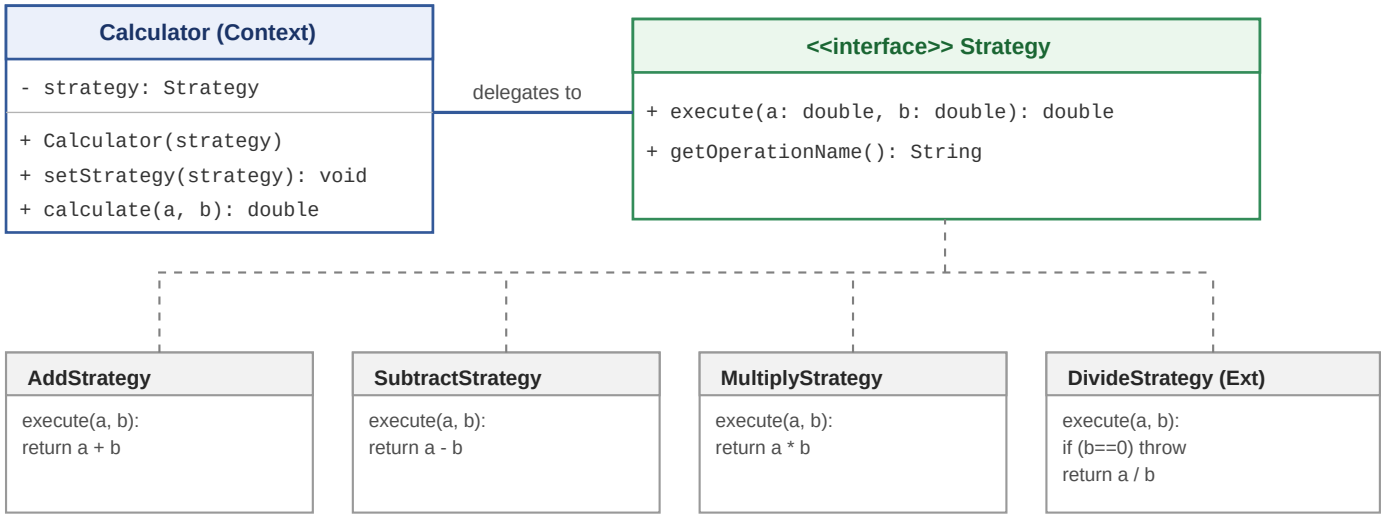
ShP-2216 Software Design Patterns | School of Software Engineering | Astana IT University

Student Name:	Alibi Maksut	**Academic Group:**	SE-2523
Chosen Topic:	Option B - Calculator	**Target Runtime:**	Java 17+ (IntelliJ IDEA)
Repository URL:	https://github.com/M1ska2021/SDPP	**Submission Date:**	26.10.2026, 11:59 PM

1. Introduction - Chosen Domain & Algorithm Family

For this assignment, Option B (Calculator) was implemented. In standard arithmetic computing systems, operations are often implemented through a long conditional switch/if-else ladder. This violates Open/Closed and Single Responsibility principles: adding a new operator requires modifying the dispatcher.
The Strategy pattern solves this by encapsulating each arithmetic rule (AddStrategy, SubtractStrategy, MultiplyStrategy, DivideStrategy) behind a common Strategy interface, delegating execution to interchangeable algorithm objects inside the Calculator context without any conditional logic.

2. UML Class Diagram & Architecture Mapping



3. Technical Requirements Compliance Matrix

Component	Formal Requirement	Implementation Defense
Strategy Interface	Declares common execution method	`Strategy defines execute(double a, double b)`
Concrete Strategies	>=3 interchangeable algorithm variants	`AddStrategy, SubtractStrategy, MultiplyStrategy + Divi`
Context	Holds strategy ref & delegates execution	`Calculator delegates without conditionals; supports set`
Client	Configures strategy; no if/switch in context	`Main configures concrete strategies directly; Context h`

4. Clean Code Requirements & Defense (Martin, Ch. 10)

4.1 Elimination of Conditional/Branching Logic in Favor of Polymorphism

Before: Anti-pattern Procedural Switch

```
public double calculate(String op, double a, double b)
  if (op.equals("+")) return a + b;
  else if (op.equals("-")) return a - b;
  else if (op.equals("*")) return a * b;
  throw new UnsupportedOperationException();
}
```

After: Polymorphic Delegation (Calculator.java)

```
public double calculate(double a, double b) {
  return strategy.execute(a, b);
}
this.strategy = Objects.requireNonNull(strategy);
// Completely branch-free and algorithm-agnostic
```

4.2 Meaningful, Intention-Revealing Names

Intention-Revealing Signatures

```
public interface Strategy { double execute(double a, double b); String getOperationName(); }
public class AddStrategy implements Strategy | public class SubtractStrategy implements Strategy
public class Calculator { public void setStrategy(Strategy strategy) { ... } }
```

4.3 Single Responsibility Principle (SRP)

Isolated Operation (SubtractStrategy.java)

```
public class SubtractStrategy implements Strategy {
  @Override public double execute(double a, double b) { return a - b; }
}
```

4.4 Context Remains Algorithm-Agnostic

Decoupled Context (Calculator.java)

```
public class Calculator {
  private Strategy strategy;
  public Calculator(Strategy strategy) { this.strategy = Objects.requireNonNull(strategy); }
```

4.5 Easy Extensibility Demonstrated (Open/Closed Principle)

Independent Strategy Addition (extension.diff)

```
+++ b/src/DivideStrategy.java
+public class DivideStrategy implements Strategy {
+  public double execute(double a, double b) {
+    if (b == 0) throw new ArithmeticException("Division by zero");
+    return a / b;
+  }
+}
```

5. Strategy vs. State Pattern Comparison (Lecture 7)

Criterion	Strategy Pattern (Applied Here)	State Pattern (Alternative)
Intent	Encapsulates interchangeable algorithmic logic	Encapsulates state-dependent object behavior
Coupling	Strategies are completely decoupled and unaware of each other	Concrete states frequently know and reference sibling states
Transitions	Client or external caller triggers strategy replacement	States autonomously initiate transitions on the Context object
Domain Fit	Operations are independent calculations selected by user	Inappropriate: Addition does not transition to Multiplication

6. Test Suite & Verification (T1-T7)

Test	Verification Target	Demonstration Output	Status
T1	AddStrategy Execution	`10 + 5 = 15.0`	**PASS**
T2	SubtractStrategy Execution	`10 - 5 = 5.0`	**PASS**
T3	MultiplyStrategy Execution	`10 * 5 = 50.0`	**PASS**
T4	Runtime Strategy Switch	`sameInstance=true | Add -> Multiply`	**PASS**
T5	DivideStrategy Extension	`10 / 5 = 2.0`	**PASS**
T6	Invariant Null Rejection	`Constructor & setter reject null strategy safely`	**PASS**
T7	Input Boundary Defense	`Division by zero throws ArithmeticException`	**PASS**

Overall Test Summary: 7 / 7 Demonstration Checks PASSED (100% Success Rate)

7. Conclusion - Flexibility Gained vs. Classes Added

Applying the Strategy pattern replaces procedural branching with polymorphic dispatch. While class count increases, the benefits are paramount:

1. Dynamic Runtime Interchangeability: Operations are hot-swapped on the same Calculator context instance.
2. Zero Regression Risk: New mathematical operations are added as separate files without modifying tested code.
3. Independent Testability: Each arithmetic algorithm is isolated and easily unit tested.

8. Academic References

1. Freeman, E., & Robson, E. (2021). Head First Design Patterns (2nd ed.), Ch. 1: Strategy Pattern. O'Reilly.
2. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship, Ch. 10: Classes. Prentice Hall.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
4. Lecture 7 Slides: Strategy Pattern Components and Comparison with the State Pattern. Astana IT University LMS.